



RepliGuard: Policy-Driven Replica Management Framework for Protecting against Acoustic Attacks

Jennifer Sheldon*
University of Florida
Gainesville, USA
jsheldon@ufl.edu

Yungwoo Ko*
University of Florida
Gainesville, USA
y.ko@ufl.edu

Sri Hrushikesh Varma
Bhupathiraju
University of Florida
Gainesville, USA
bhupathirajus@ufl.edu

Sara Osmanovic
University of Florida
Gainesville, USA
s.osmanovic@ufl.edu

Weidong Zhu
Florida International University
Miami, USA
weizhu@fiu.edu

Md Jahidul Islam
University of Florida
Gainesville, USA
jahid@ece.ufl.edu

Sara Rampazzi
University of Florida
Gainesville, USA
srampazzi@ufl.edu

Abstract

With the rapid expansion of data center infrastructure driven by the AI boom and the growing energy demands, companies have begun deploying underwater data centers, leveraging water's natural cooling properties. While promising in terms of sustainability and efficiency, underwater environments introduce unique security vulnerabilities, particularly susceptibility to acoustic attacks targeting storage systems. Proposed defense strategies are limited to disk-level detection, while system-level mitigations remain absent. In this work, we present RepliGuard, a novel vulnerability-aware policy-driven replica management framework integrated with CockroachDB. Our system combines OS-level detection using Extended Berkeley Packet Filter (eBPF) probes with a policy-driven replica relocation scheme to mitigate the attack impact without requiring low-level disk access or sacrificing node functionality. Our LSTM-based model achieves a true positive rate of 99.4% and a true negative rate of 99.1% when detecting subtle acoustic attacks against hybrid SDD cache-based architectures, and takes approximately 0.5 seconds to detect subtle attacks with an inference latency of 0.27 ms per prediction compared to 30-second detection of previous works. Although CockroachDB's automatic replica rebalancing scheme fails to mitigate the attack and instead increases the cluster's P99 SQL service latency by 215% in our testbed, our proposed mitigation dynamically reassigns replicas to unaffected nodes, reducing the attack impact by approximately 83% while preserving node functionality and activating within 2 seconds. This work demonstrates the first vulnerability-aware, system-level strategy to ensure resiliency against subtle acoustic attacks.

*Denote co-first authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.
SecDev '26, Montreal, QC, Canada
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2602-6/2026/07
<https://doi.org/10.1145/3805773.3806000>

CCS Concepts

• Security and privacy → Database and storage security; • Computing methodologies → Distributed computing methodologies.

Keywords

Critical Infrastructure Security, Underwater Data Center, Distributed Systems

ACM Reference Format:

Jennifer Sheldon, Yungwoo Ko, Sri Hrushikesh Varma Bhupathiraju, Sara Osmanovic, Weidong Zhu, Md Jahidul Islam, and Sara Rampazzi. 2026. RepliGuard: Policy-Driven Replica Management Framework for Protecting against Acoustic Attacks. In *Proceedings of the 2026 ACM Secure Development Conference (SecDev '26)*, July 5–6, 2026, Montreal, QC, Canada. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3805773.3806000>

1 Introduction

With the recent AI boom, data center infrastructure is expanding rapidly [68]. However, on-land data centers have sparked concerns regarding noise and air pollution [27, 55], water consumption [78], and land use [42]. More importantly, increasing power consumption has heightened concerns, providing evidence of growing strain on energy infrastructures [40]. To address these issues, companies such as Network Ocean [56], Highlander [53], and Subsea Cloud [38] have deployed data centers underwater to leverage water's cooling properties, achieving more than 30% reduction in power consumption [80]. Although these emerging infrastructures show promise in alleviating energy concerns, their surroundings augment their vulnerability to certain classes of attacks. Previous works [9, 69] have established that hard disk drives (HDDs), typically used as back-end storage, are inherently susceptible to acoustic interference, and external vibrations can displace their read/write heads and cause I/O failures. This vulnerability is significantly amplified in the underwater environment [1, 8, 46, 73, 74, 81]. This amplification occurs because sound waves propagate through water four

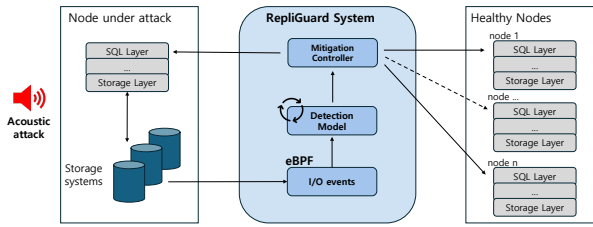


Figure 1: Overview of the RepliGuard scheme functioning. Our detection model observes I/O probe events and alerts the mitigation controller when it detects acoustic attacks. The controller reconfigures the replica placement and instructs the data center nodes to mitigate the attack impact.

times faster than air and are converted into mechanical vibrations in solid structures. These vibrations travel through internal server structures, reaching the storage system, ultimately leading to I/O operation failures.

Researchers have shown that regulating the energy of the acoustic signal allows attackers to time out file system processes and cause application failures in under 85 seconds, up to 6 meters away from the submerged server enclosure, using only a commercial swimming pool speaker. Furthermore, it is possible to manipulate virtual machine resource allocation and I/O latencies in distributed databases, despite the presence of fault-tolerant RAID configurations and SSD-based cache architectures.

Another distinctive characteristic of these attacks is their ability to cause permanent damage to the storage system, even with low-energy attacks, without showing immediate visible consequences. This is particularly problematic for underwater data center deployments, where replacing failed disks in submerged environments is extremely difficult and impractical (e.g., HiCloud’s submerged data centers are deployed 35 m below the sea surface [39]). Previous works have suggested machine learning-based mechanisms to detect such acoustic attacks, based on throughput variation [73] and storage system profiling [8]. The methodologies employ K-means clustering to detect anomalies above a certain threshold; however, these strategies are limited by the following factors: (1) They rely on storage-level information such as in-depth low-level disk profiling or the extraction of the Position Error Signal (PES) [8] data directly from each hard disk. Both approaches require accurate disk profiling and frequent updates as storage systems wear or are replaced, which is not cost-effective. Furthermore, PES data are rarely available due to proprietary HDD firmware restrictions, which limit access to low-level diagnostic data [66]. (2) Only detection mechanisms are considered; thus, it remains unexplored how data centers can recognize subtle attacks without low-level hardware information and simultaneously mitigate the attack consequences, maintaining reasonable performance and node functionality. Furthermore, it remains unclear if these defense strategies can be effectively used to protect critical processes, such as replica management, whose compromise can directly undermine consistency guarantees and the reliability of cloud environments.

Our work focuses on starting to address this research gap by providing **RepliGuard**, a policy-driven replica framework which integrates with CockroachDB [16], a widely-adopted distributed database used by companies such as SpaceX [20] and Netflix [21]. The framework illustrated in Figure 1 detects acoustic attacks and automatically regulates replica management without the need for single-disk profiling and quickly redirects the critical I/O path, thereby mitigating the attack effect without causing automatic node drop or losing node functionality.

Our contributions can be summarized as follows:

- We provide a novel framework composed of an OS-level detection scheme and a distributed database’s policy-driven replica management system designed to automatically relocate replicas, removing the affected nodes from the critical I/O path. Based on the severity of an attack, our strategy partially unloads the replica that the affected nodes manage.
- Our novel detection model based on LSTM networks leverages eBPF data accessible to the OS to allow detection of subtle attacks that remain undetected by conventional automatic node protections, achieving a true positive rate of 99.4% and a true negative rate of 99.1% with a 0.5-second latency overhead in our proof-of-concept implementation.
- Our proposed policy-driven replica management scheme, designed to integrate with CockroachDB, dynamically re-routes replicas from nodes under acoustic attacks, based on the severity of the attack. Our evaluation shows a latency overhead of between 1.8 and 2.0 seconds while restoring the degraded system’s I/O performance with a maximum recovery rate of approximately 80%.¹

2 Background and Related Works

2.1 Storage Systems in Data Centers

Major cloud service providers, including Google [81], Alibaba [44, 46], and Meta [54], employ heterogeneous storage architectures in their data centers that combine full-HDD deployments with hybrid configurations such as SSD-based caching layered over HDD backends. Despite the growing use of SSDs, it is estimated that HDDs currently store more than 90% of data in data centers [65] and are projected to still account for 80% by 2028 [32]. Although popular SATA SSDs generally outperform HDDs in terms of speed, they exhibit a high rate of *infant mortality*, meaning an increased likelihood of early-life failures, particularly under write-intensive workloads [51]. NVMe SSDs, on the other hand, are more likely to suffer from *grey failures*, where devices show anomalously high latencies, compared with HDDs [47]. For these reasons, data center deployments aim to balance cost efficiency, storage capacity, and performance by leveraging the high throughput and low latency of SSDs for frequently accessed data while maintaining HDDs for large-scale persistence.

Underwater data centers utilize essentially the same storage technology as conventional land-based data centers, packaged within specialized subsea vessels able to withstand underwater pressure while protecting the hardware from corrosion and moisture. For

¹Code and artifacts are available at <https://zenodo.org/records/19357661>

example, the Beijing-led Hainan underwater data center [30] currently contains 24 racks with up to 400 high-performance servers, which employ conventional HDD-SSD hybrid storage technologies.

2.2 Attacks Against Underwater Servers

Recent research has shown that acoustic injection attacks against storage systems and servers deployed in submerged metallic enclosures are feasible [1, 8, 73, 74]. Malicious sound waves propagate through water and vibrate submerged structures and their internal servers. By controlling the energy of the acoustic signal, attackers can induce DoS, manipulate performance, and permanently degrade disks. Previous works have explored such attacks in the context of denial-of-service and manipulation of distributed databases, file systems, and resource management. For example, Sheldon et al. [73] demonstrated that adversaries can use volume-modulated acoustic waves to manipulate the latency of distributed databases by targeting a subset of server nodes, blocking access to HDFS filesystems, and forcing virtual machine reassignment between nodes. They also provide an ML-based classification mechanism to detect acoustic injection, which captures changes in the rate of data transferred in close-proximity disks, based on the assumption that spatially located disks are affected simultaneously by the attack. Abdullah et al. [1] build upon this by proposing the use of autonomous robots to surveil data center surroundings to detect and localize attackers. Blow et al. [8] instead propose the monitoring of the position error signal (PES), which measures the deviation of the disks' read/write head from the centers of their tracks. However, these defensive strategies suffer from two major limitations: (1) they require low-level disk profiling using often firmware-restricted data, and (2) focus only on detecting anomalies, with no consideration of mitigation strategies after detection.

In this work, we present a novel mitigation framework integrated with CockroachDB that ensures node resiliency even against subtle attacks that do not noticeably affect throughput or trigger automatic protection mechanisms while silently damaging storage systems. Specifically, we concentrate our attention on replica management and data replication as a critical component for maintaining data availability, integrity, and fault tolerance in cloud environments.

2.3 Data Replication in Distributed Databases

Data centers employ data replication for multiple purposes such as improving fault tolerance, reducing latency, and recovering from disasters [36]. The number and location of replicas are configured to optimize metrics such as performance, availability, reliability, and energy efficiency [70]. These replicas are typically distributed to servers in the same data center [24] or across geographic regions [23]. To maintain consistency between replicas, geo-distributed databases in particular employ consistency models that provide certain guarantees (e.g., reads from all replicas will reflect the latest updates) depending on specific requirements of the application. For instance, in global financial systems [22] or real-time inventory management, stale data can lead to severe consequences such as double-spending, overselling, or violating transactional integrity. Inconsistencies can cause incorrect balance calculations, race conditions between concurrent updates, or the propagation of outdated state across regions. Such failures not only

compromise the correctness of business operations but can also trigger cascading effects such as financial losses, regulatory breaches, or the need for costly reconciliation procedures.

In these critical domains, distributed databases such as CockroachDB [16] often serve as the core infrastructure, handling the precise management of the replicas.

CockroachDB Replica Management. In this work, we focus on CockroachDB, a widely adopted distributed SQL database system in data centers [12, 76], as a case study for implementing our mitigation strategy. In CockroachDB, replica management is handled by an internal component known as the *allocator*, which runs on each node [16]. Upon a write request, the target key is mapped to a specific range. While the request is routed to the replicas responsible for that range (typically a Raft group [58]), the allocator is responsible for determining the physical placement of these replicas across the distributed systems. It also continuously evaluates the status of its ranges based on cluster-wide gossip information to ensure that each range maintains the default replication factor (typically three) and selects appropriate nodes to store the replicas based on specific criteria. Criteria can include maximizing diversity across domains or leveraging load-balancing algorithms such as those balancing based on queries-per-second, which represent how frequently requests are given to that range [19]. Furthermore, the allocator dynamically relocates stored replicas when rebalancing is necessary, such as when a node becomes unresponsive (heartbeat failure), accumulates an excessive number of replicas, or experiences high resource utilization [17].

CockroachDB runs client-requested queries through layers. The SQL layer converts SQL statements into Key-Value (KV) operations [16]. The transaction layer runs those operations with ACID (Atomicity, Consistency, Isolation, and Durability) [15] guarantees. After that, the distribution layer receives the flow of KV operations from the transaction layer and sends them to the proper node's replication layer. The default replication factor for CockroachDB is three, and Raft Consensus is applied at the replication stage. A majority consensus of replica nodes is required for a write request to be committed on the cluster. Finally, the Pebble Storage Engine [50] performs the KV operations on the storage disks. Specifically, the data is stored as KV pairs in one monolithic sorted key space. This key space is split into contiguous ranges, so every key belongs to exactly one range. Each range is then replicated across nodes in a Raft group. One replica holds the range lease; that replica (called the *leaseholder*) is also the Raft leader and coordinates the requests for that range.

When requesting a write operation to a CockroachDB cluster, the client is routed to the leaseholder node. The leaseholder proposes the write, and it commits when a majority of voting replicas append the entry to their Raft logs. Read operations are also by default handled by the leaseholder, but they do not need a Raft consensus. The administrator can configure the lease preferences, which drive the lease holder placement to prefer certain nodes, or establish constraints on the replica distribution, which prohibits the placement of a replica on a specific node. The Pebble storage engine is based on RocksDB, thus it uses a Write-Ahead Log (WAL) and Sorted String Table (SSTable) to read and write the data blocks to disk [50]. New updates are first persisted to the WAL and later flushed/compacted into SSTables. Because commits require a Raft

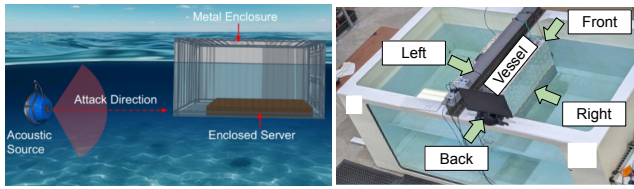


Figure 2: (Left) Attack overview. (Right) Real-world testbed used in this work for the acoustic attack evaluation with marked sound source directions.

quorum and durable logging, end-to-end latency often comes from network round-trips to a majority of replicas and from WAL's fsync latency (the time it takes for WAL to be written on disk) of those replicas. Thus, if a certain node stalls due to a software or hardware malfunction, the replica belonging to the stalled node gets affected, and requests on that replica will also be delayed.

3 Threat Model and Attack Overview

This work addresses the detection and mitigation of subtle acoustic attacks targeting servers deployed in underwater enclosures. Unlike strong acoustic attacks explored in previous works [73] that can cause an immediate denial of service (e.g., > 34 dB Δ SPL²), these threats manipulate system behavior while keeping performance within acceptable thresholds, all while causing permanent damage to the underlying storage systems.

In this context, we consider a budgeted attacker who aims to affect the accessibility and availability of critical data (e.g., stock market, military, government intelligence, medical) housed in geo-distributed databases by indirectly undermining their replica management system. This can be achieved by performing low-energy, prolonged acoustic attacks (e.g., 26 to 32 dB Δ SPL) that target one or multiple server nodes located in the submerged enclosure.

Acoustic Attack Implementation. As described in previous works [8, 73, 74] and shown in Figure 2, the attack can be mounted using commercial underwater speakers or Long Range Acoustic Devices (LRADs) [29] to generate long-range attacks (meters to kilometers away from the source), which can be attached to a boat or a more sophisticated underwater robot. The attacker transmits the acoustic waves at a frequency that matches the natural resonant frequencies [7] of the metallic enclosure. This frequency can be identified as described in previous work by performing a frequency sweep and observing throughput variations in benign applications running on the victim servers.

The resulting sound-induced mechanical vibrations propagate to the metallic enclosure containing the servers and the storage systems, causing the actuator arm of HDDs to move outside the disk track's limits, in turn increasing I/O operation latencies on the affected disks [9]. The attacker can control the volume of the injected sound wave to augment or reduce the attack energy, and the level of storage degradation is proportional to the attack energy, affecting the latency of write operations, as shown in Figure 5.

²The notation "dB Δ SPL" refers to the decibel sound pressure level (SPL) above the environmental noise level captured with a hydrophone.

Attacker Knowledge and Assumptions. Following the assumptions made in previous works, the adversary has no physical access to the underwater infrastructure or servers and cannot tamper with any hardware, software, or network communications. We assume that the attacker has knowledge of the location of the victim's submerged data center and can generate acoustic waves with sufficient volume at a resonance frequency of the target structure (e.g., the 5.1 to 5.3 kHz range). Finally, we assume the data center uses a geo-distributed database cluster (CockroachDB in our case study) that utilizes replication for fault tolerance.

Testbed and Experimental Setup. In this work, we use a commercial Clark Synthesis AQ339 Diluvio underwater speaker [13] similar to that used in previous works [74]. We place the speaker in a $1.2 \times 3.0 \times 1.5$ m water tank facing a partially submerged $0.9 \times 0.6 \times 0.5$ m metallic enclosure containing the victim server node as shown in Figure 2. To characterize performance changes during attacks, we perform the acoustic injection by transmitting a tone at the resonance frequency of our structure (3.1 kHz). We run our evaluation on 3 ASRock Rack 1U4LW-X570/2L2T RPSU servers [52] and a Supermicro CSE-823 rack server [75] in a LAN. This experimental setup is consistent with previous works [8, 73, 74]. We also simulate the attack in the air using a Walfront [28] resonance speaker vibrating the servers.

3.1 CockroachDB Under Attack

As described in Section 2, geo-distributed database systems automatically remove nodes if they become unresponsive for prolonged periods of time (heartbeat failure) [22, 35, 82]. This procedure is necessary to remove failing nodes that act as bottlenecks. Unfortunately, such remediation does not mitigate acoustic attacks, particularly if they are subtle and do not trigger removal mechanisms.

To better understand the impact of the attack on the overall system latency, we use CockroachDB as a representative case study. The acoustic attack is simulated by injecting synthetic delays, which we refer to as "attack-induced delays", into a CockroachDB storage node, varying the induced delay to represent different levels of acoustic signal intensity. We then evaluate the per-node performance and how the attack effect propagates to the rest of the cluster through CockroachDB's replica consistency mechanisms (Figure 3). Our analysis reveals two key points:

(1) CockroachDB's node removal mechanisms do not defend against subtle acoustic attacks. Despite CockroachDB's self-healing protocols, our experimental results reveal that acoustic attacks can induce significant performance degradation while remaining below the threshold required to trigger node eviction. Figure 3(a) shows the average WAL Fsync latency with its distribution when the attack is posed on the victim node. As the injected delay increases, which corresponds to higher acoustic signal strength, the victim node's latency rises significantly. However, the attack remains hidden, successfully degrading the system without being detected or terminated by the cluster's protection logic.

(2) Performance bottlenecks in a single victim node propagate throughout the system. We further evaluate the SQL service latency across the cluster, comparing both compromised and benign nodes as shown in Figure 3(b). The results demonstrate that the performance of the geo-distributed service is restricted by the

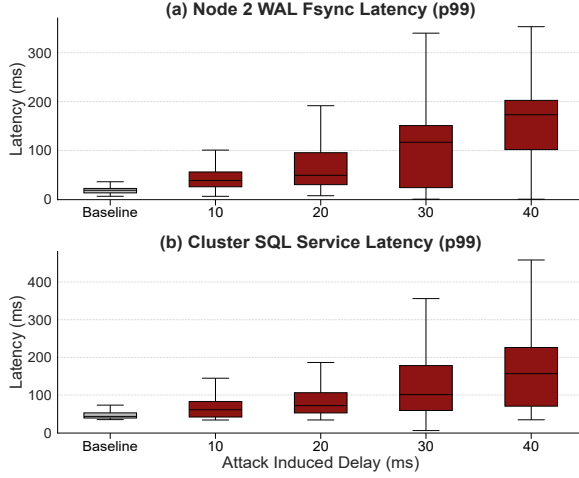


Figure 3: The gray box plot represents the baseline. (a) Node 2's (victim node) WAL fsync latency increases. (b) A single attacked node (node2) becomes the bottleneck of the system and affects the overall cluster's SQL service latency (attack triggered after 900 seconds).

least-performant node. Specifically, when node2 is compromised, the latency observed at all unaffected nodes converges with the degraded latency of the victim node. This confirms that an acoustic attack, even when confined to a single storage node, can effectively slow down the entire database due to the synchronous constraints of distributed consensus.

Slow-Fault Defenses. Previous research on slow-fault handling in distributed systems, such as IASO [60] and Copilot [57], also relies on static thresholds to detect performance degradations, leading to similar undetectability under adversarial attacks. Recently, Lu et al. [45] have proposed an Adaptive Detection at Runtime (ADR), a library that replaces static thresholds with adaptive, percentile-based detection. However, ADR mitigation actions still depend on each target system's built-in mechanisms (i.e., WAL log rolls in HBase, node exclusion in CockroachDB), none of which provide replica-level relocation to remove degraded nodes from the critical I/O path. RepliGuard addresses both detection and mitigation: it detects subtle acoustic attacks through OS-level eBPF probes and mitigates their impact by dynamically relocating replicas away from affected nodes using policy-driven zone constraints.

4 Framework Design

In this section, we describe our proof-of-concept framework composed of a detection model and a policy-driven replica management system as illustrated in Figure 1. The detection component leverages a Long Short-Term Memory (LSTM) network that analyzes Linux OS tracepoint probes of I/O events in the node cluster to detect whether an acoustic attack has occurred. The policy-driven replica management scheme integrated with CockroachDB dynamically relocates a portion of replicas upon attack detection while maintaining the continuous operation of the distributed database. This mitigation occurs in three steps, as shown in Algorithm 1: (1) the

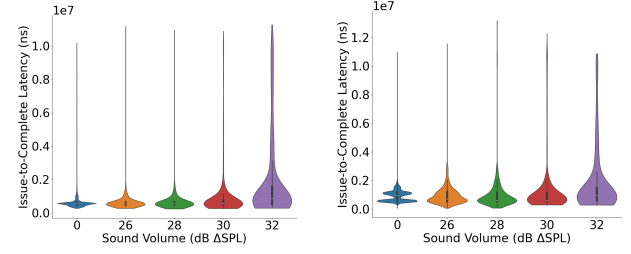


Figure 4: Violin plots of the issue-to-completion latencies for (Left) random and (Right) sequential write workloads based on the FIO benchmark [6].

detection system identifies attacks based on anomalous I/O probe events. (2) To enforce partial isolation, the controller retrieves a pre-defined isolation ratio (r) corresponding to the detected attack level. It then calculates a split key (K_{split}) based on the isolation ratio and executes SQL commands to split the table range into two logical partitions: a retention partition (P_{stay}) and a migration partition (P_{move}). (3) Finally, the controller applies Zone Configuration constraints exclusively to the migration partition (P_{move}). By enforcing negative constraints (e.g., constraints: $[-N_{vic}]$) and updating lease preferences, the system forces CockroachDB's internal allocator to migrate only the replicas within P_{move} to healthy nodes. This effectively reduces the load on the victim node by the pre-configured ratio while preserving its remaining functionality.

4.1 Detection Model

Extended Berkeley Packet Filter (eBPF) programs [25] are extensively used in cloud environments by providers such as Adobe, Alibaba Cloud, and AWS [37], to improve observability and monitor network traffic and performance [2, 11]. Our detection solution leverages tracepoint information logged using eBPF probes. Different from previous detection methods [8, 73] which use hard disk-level proprietary signals (e.g., PES), or disk profiling, this strategy allows for detection using data available at the OS level. These probes typically provide information about the timing and properties of I/O events (e.g., device sector number and size of a request). In our strategy we consider four events as they are directly correlated to the attack influence on the latency of write operations: the start of processing a given request (i.e., `tracepoint:block-io_start`), the insertion of the request into a device hardware queue (`tracepoint:block:block_rq_insert`), the instant that a queued event is issued to a device driver (`tracepoint:block:block_rq_issue`), and the instant that a portion of the request has been registered as completed (i.e., `tracepoint:block:block_rq_complete`).

We represent these four events as three latencies: request-start-to-queue-insertion, queue-insertion-to-drive-issue, and issue-to-completion. By considering these sub-latencies rather than the full latency from application I/O request to completion, we minimize the influence of external factors not related to the acoustic attack effects, such as delays incurred by I/O scheduling.

LSTM Network. Previous work detection mechanisms rely on identifying attacks based on a fixed threshold for the number of disks exhibiting anomalous throughput or PES. Such approaches,

however, overlook subtle attack scenarios in which performance degradation is deliberately small or diffuse, allowing the attack to remain undetected. In particular, disk latency distributions perturbed through acoustic attacks can vary depending on the attack severity and workload, as shown in Figure 4. In our testbed, applying an attack with volumes increasing from 0 to 32 dB Δ SPL (following the low-energy attack scenario of previous work [73]), the mean issue to completion latency for a random write workload increases from 0.6 to 1.9 ms and the standard deviation increases from 0.3 to 2.6 ms for a full-HDD server architecture. For hybrid architectures, the SSD cache causes fluctuation, with the same workload increasing the mean issue to completion latency from 1.7 to 42.6 ms and the standard deviation from 7.6 to 894.3 ms. This is due to the periodic flushing that floods the backend storage devices, and the standard deviation increases because the severity and number of outliers grow proportionally with attack volume.

To address this challenge, we develop a novel LSTM network that reliably detects behavioral differences under low-energy acoustic attacks, informing our policy-driven replica management about affected nodes. Prior work has demonstrated that LSTMs can effectively capture subtle variations in system performance metrics, making them widely applied for anomaly detection and natural failure prediction [31, 33, 63]. A comprehensive comparison of machine learning approaches and the justification for our LSTM model selection is provided in Appendix A.

Based on these observations, we design our model to receive as input sequences of the three latencies derived from the eBPF probe data, divided into fixed-length windows of 5 (approximately 0.5 seconds of disk operations). This enables the model to capture request latency information from all disks, and it is optimized with a sufficiently short duration to allow detection while preserving short-term temporal dynamics. The architecture comprises two stacked LSTM layers to learn hierarchical temporal dependencies, followed by a dense output layer with a tanh activation function to classify anomalous behavior. The network is then trained on our collected data, using the Adam optimizer [41] with a mean squared error (MSE) loss function.

4.2 Policy-Driven Replica Management

In Section 3.1, we show how a single server node affected by a low-energy attack can influence the latency of the whole cluster. Through the allocator and the lease manager, CockroachDB systems attempt to rebalance the leases and replicas; however, the cluster cannot recover from the attack, keeping processing requests while the storage devices are damaged.

Considering M attacked nodes, the goal of our replica management system is to quickly remove them from the system's critical I/O path. Nodes are in the critical I/O path when they are lease holders or voting replicas for multiple ranges. By default, each range in CockroachDB has three replicas on distinct nodes ($r = 3$). To maintain 3 replication while excluding M attacked nodes from replica placement, we assume the cluster contains at least $M + 3$ nodes (i.e., ≥ 3 healthy nodes). As a proof-of-concept example, we consider our cluster of 4 nodes, and we assume one node is under attack ($M = 1$). Because CockroachDB provides range-level replication, a lease-based coordinator, and general placement policies (e.g., lease

Algorithm 1: Mitigation via Dynamic Partitioning

Input: Attacked Node N_{vic} ; Database D ; Partial Ratio r
($0 \leq r \leq 1$)

Output: Partially mitigated traffic on N_{vic} via partitioning

- 1 Fetch key range $[K_{min}, K_{max}]$ from D
 - 2 Calculate split key $K_{split} \leftarrow K_{min} + \lfloor (K_{max} - K_{min}) \times r \rfloor$
 - 3 Run SQL (ALTER TABLE $D.kv$ SPLIT AT VALUES (K_{split}))
 - 4 Run SQL (ALTER TABLE $D.kv$ PARTITION BY RANGE (k)
(PARTITION P_{stay} VALUES FROM (MINVALUE) TO
(K_{split}), PARTITION P_{move} VALUES FROM (K_{split}) TO
(MAXVALUE)))
 - 5 Construct lease preferences $\mathcal{L}_{safe}$ excluding N_{vic}
 - 6 Run SQL (ALTER PARTITION P_{move} ... CONFIGURE ZONE
USING constraints = $[-N_{vic}]$, lease_preferences = $\mathcal{L}_{safe}$)
-

preferences and constraints), we design our policy-driven strategy by directly leveraging built-in mechanisms and I/O redirection as illustrated in Algorithm 1.

The core idea of our policy-driven strategy is to reroute a partial amount of work from the victim node to the other healthy nodes, depending on the attack severity. Considering the subtle level of attacks, it is not necessary to fully exclude the victim node from the workload. Instead, we control the appropriate amount of work that the node can handle to maintain its functionality. If a severe attack is detected, we retrieve most of the work from the victim node. This approach is possible as CockroachDB supports a single large key space. Each key (which is translated from the client's request) is mapped to exactly one range, and this range is handled as a replica by three different nodes.

CockroachDB provides lease preferences and zone constraints configuration options that serve as policy-based placement rules for leases and replicas. To provide a case-by-case effective mitigation strategy based on the severity of an attack, we divide the entire key space for a uniformly random database workload into two partitions. Specifically, one partition is subject to the policy, while the other remains unaffected. The policy-applied partition is P_{move} , and the unaffected partition is P_{stay} . We give the lease preferences to the unaffected nodes, while setting zone constraints (i.e., constraints on replicas) for the attacked node. Based on this, we ensure that leases do not automatically migrate back after relocation, and replicas are relocated to the other nodes by the allocator and lease manager. This ensures that new incoming ranges will not be influenced by the attacked node. In our evaluation, we determine the optimal P_{stay} ratio based on our empirical data and the tested workload.

5 Framework Implementation

In this section, we discuss the implementation of our detection and mitigation framework, including the details of data collection, simulations, and policy-driven replica management.

Probe Data Collection. To test and train our detection model, we use the eBPF program compiler bpfftrace [62] to collect benign probe logs from 2 35-minute trials of the KV benchmark [43]. This allows

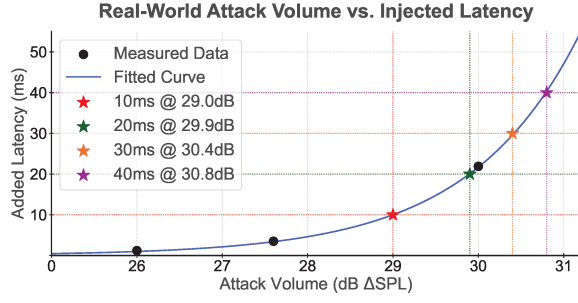


Figure 5: Correspondence of dB Δ SPL values from real-world attacks to our simulated injected latency.

us to collect the request-start-to-queue, queue-insertion-to-drive-issue, and issue-to-completion latencies described in Section 4. To determine whether our attack succeeds at different degradation severities, we collect the KV benchmark data for five levels of degradation (10, 20, 30, 40, and 50% degradation in data transmission rates compared to the baseline), calibrated based on the FIO random write workload [6]. As shown in Figure 4, the latencies used in our detection scheme increase with increasing volume. Our 10 – 50% degradations correspond to 2, 3.25, 3.42, 3.423, and 3.437 VPP amplitudes on the function generator that control our resonance speaker vibrations. Performing repeated attacks on the same system can leave it in an unstable state. To minimize such lingering effects, we perform a hard reboot on the server (i.e., turn the power off and on), recreate the RAID 0 device and cache architecture, and warm up the cache between each trial. Following this methodology, we collect 1000 request samples (with variation depending on the cache hit and miss rate and the disk mapping for the random workload) for each of the four disks for each volume under the KV workload.

Tiered Storage Systems Simulation. To simulate the tiered storage systems typically deployed in data centers [67], we utilized the Open CAS library [59] to construct a hybrid storage architecture. Since we are equipped with three identical servers (victim node included) and one desktop, we applied a different storage setup script to each of them. First, all the nodes configure the 5 GB of SSD partition as a cache device using Open CAS on top of the HDD-based RAID 0 device. However, the desktop is not equipped with HDDs, thus we utilized the Linux cgroup v2’s io controller to mimic the HDD [26]. For each NVMe partition in the desktop node that simulates a single HDD, we set the read and write maximum bandwidth and read and write iops to match those of the other HDDs in the servers. Consequently, NVMe SSD cannot perform above that io.max bar, bounded by the HDD performance.

Attack Simulation. We use real-world data collected from our testbed, as described in Section 4, to simulate an attack on the replica management system. We utilize the Linux Device Mapper framework [79], leveraging the *dm-delay*, which maps a virtual block device to physical storage via a mapping table and transparently delays read and write operations by a specified duration. We allocate one of the nodes as the victim node and insert a device mapper layer on top of each HDD partition of the victim node. Initially, this layer is configured with the *dm-linear* target, mapping

I/O requests directly to the underlying physical devices with negligible overhead. To trigger the attack, we dynamically swap the mapping table from *dm-linear* to *dm-delay* without unmounting the file system or restarting the database process. In this way, the *dm-delay* stays on top of the actual block device, and a pre-defined latency is added to every I/O that passes the *dm-delay* and then the actual block device. This approach adds a static, configurable extra latency (e.g., a 10 ms latency), defined as attack-induced delay, to every read and write operation, mimicking an acoustic attack that hinders I/O operations on HDDs. We selected four increasing acoustic volumes (dB SPL) to mimic the attack each of which is converted into four different attack induced delay (10 ms to 40 ms) to the HDD’s I/O performance. In Figure 5, black dots represent the real-world attack volumes’ effect on latency measured in our testbed. Based on these values, we interpolated our four different levels of attack-induced delay. Finally, to simulate a scenario where the SSD cache is sufficiently filled and not empty, we use data pre-inserted into the DB, provided by the workload initialization phase in CockroachDB [16].

Policy-Driven Replica Management. We utilize SQL statements (e.g., `ALTER PARTITION  $P_{move}$  CONFIGURE..`) to apply our policy-driven replica management strategy. We control lease preferences and replica constraints using SQL statements with the CockroachDB SQL execution client [16]. To process the mitigation (replica scheme) in the simulated environment, we ran experiments with fixed configurations. When the SQL requests generated by the workload fill the CockroachDB ranges, each range grows in size. When the range is filled to the default size (512 MiB), it splits into two ranges. Since our KV workload creates uniformly random keys, existing ranges are uniformly filled as the workload runs. Thus, if we retain the workload configurations for every experiment (e.g., size of block, level of concurrency), we can expect the range splits to be triggered at the same point in time.

We fixed the size of the block to 1,024 Bytes, read amount to 30%, and launched the attack from the starting point of logging when reached baseline stability. In this way, the split occurs at a consistent point in time among trials, and consequently, we set the attack to start after that time (around 900 seconds). This ensures that the system has reached a steady, consistent state for our evaluation, as the attack can delay the time before subsequent splitting occurs by slowing the rate at which range sizes increase. Through this control, we ensure that our attack does not hinder the first range split, only affecting the process between the first and second range split. Considering the time required for the detection module to detect an attack and send the result to the mitigation controller, the mitigation scheme is launched 1 second after the attack starts.

6 Evaluation

To evaluate the performance of our proof-of-concept framework, we use a desktop equipped with an Intel Core Ultra 7 265K CPU, 64 GB of RAM, two Samsung 990 Pro 2 TB NVMe SSDs, and Ubuntu 25.10 with the kernel version of 6.17, and a server with an AMD 3.7 GHz CPU and 16 GB of RAM running Ubuntu 24.04.1 on an application solid-state drive. Four Seagate Exos 7E2 1 TB SATA enterprise HDDs [64] are housed in the server with RAID 0 configuration, which is cached by a 5 GB SSD with a writeback algorithm.

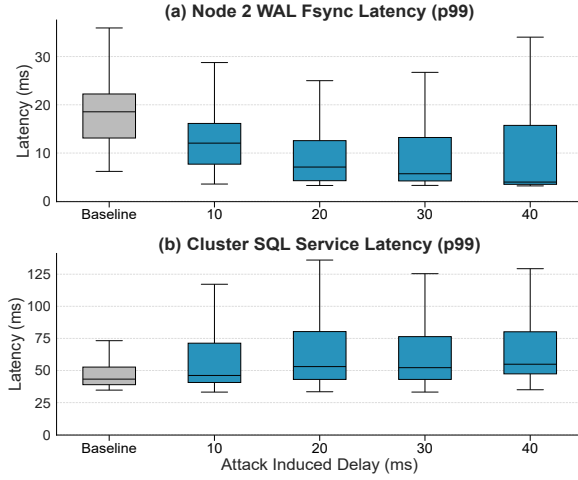


Figure 6: The gray box plot represents the baseline. (a) Node 2's (victim node) WAL fsync latency is lower compared to baseline because mitigation migrates part of the replicas, causing Node 2 to process fewer I/O requests. (b) Cluster's SQL service latency stays within a stable range when mitigation is applied (attack triggered after 900 seconds; mitigation triggered 1 second after the attack).

Testbed. The acoustic attack data used for the latency interpolation are collected in the testbed shown in Figure 2. We then use the Walfront resonance speaker [28] to collect our attack dataset.

Workloads. The eBPF probe information is collected by running the built-in KV workload in CockroachDB [16] with increasing attack amplitudes. During the evaluation of the single node attack scenario, we maintained a concurrency of 64 workers, a read ratio of 30%, and a block size of 1 KB.

6.1 Detection Evaluation

Using the latencies collected from the attack as described in Section 5, (request-start- to-queue-insertion, queue-insertion-to-drive-issue, and issue-to-completion), we train our LSTM model on 1000 randomly selected combinations of I/O request latencies and test it with 1000 samples of independent and identically distributed observations. We repeat this process for 10 trials and observe the worst-case performance of the model. We use true negative rate (TNR), true positive rate (TPR), false negative rate (FNR), and false positive rate (FPR) as evaluation metrics, which quantify the model's ability to correctly identify both attack and non-attack data. Across all considered degradation severities, the model achieves a minimum TPR of 99.4%, FNR of 0.6%, and a TNR of 99.1% with a FPR of 0.9%, indicating that our detection mechanism can achieve high performance across multiple attack severities. Inference evaluation on the Intel Core i7-12800H CPU demonstrates a maximum latency of approximately 0.27 ms per prediction. These results show that the proposed detection model achieves high accuracy with low error rates while maintaining real-time inference efficiency.

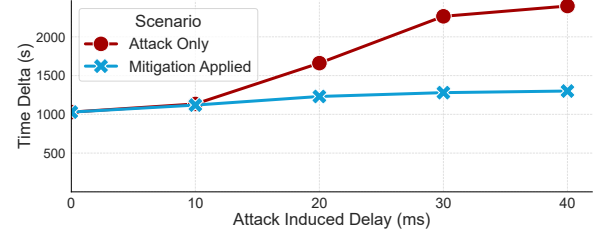


Figure 7: The range split point is delayed more as the attack-induced delay increases. With the mitigation applied (blue line), the impact of latency on the range split interval decreases (0 ms represents the baseline with no attack).

6.2 Replica Evaluation

Since WAL fsync is the critical path for the SQL write [16], clusters running write-heavy workloads are significantly affected by the attack. Figure 3 demonstrates that our simulated attack on HDDs significantly degrades the victim node's WAL fsync latency and the overall cluster SQL service latency. We average the results from three evaluation trials across four nodes to measure the SQL service P99 latency. For WAL fsync latency, we present data from the victim node alone to visualize how the simulated attack directly affects its I/O performance. The trends show that the attack linearly increases the median of SQL service P99 latency by 41% to 262%, depending on the attack induced delay. On the other hand, Figure 6 shows that our mitigation (replica scheme) effectively alleviates the attack's impact on the system. Specifically, observing the median of SQL service P99 latency, our scheme recovers between 66% and 90% of the performance degradation caused by the attack.

Another important metric for showing the effect of an attack is the range splitting point. The admission control and write stall system temporarily stalls the write when SSTable compaction takes more time than expected [18, 49]. Thus, the result of subtle attacks (e.g., those that induce only a 10 ms increase in attack-induced delay) is sometimes not directly reflected in the latency metrics. Instead, we observed the time point where the range split is triggered. As described in Section 5, in CockroachDB, each key-value (KV) pair is allocated to a specific range, and that range grows in size when a request containing a new key enters the cluster. When a range grows large enough (512 MiB, default value), CockroachDB splits it into two ranges [16]. Thus, if requests to the CockroachDB cluster are delayed and less data is accumulated within the range, the range split will also occur at a later time. We measure the time difference between the first range split that occurs before the attack and the second range split that occurs after the attack.

As shown in Figure 7, the time difference between two range split points increases as the level of attack arises. Since we are running the KV workload with the same configuration, the range split point effectively identifies the consequence of an attack. The blue line plot shows that our mitigation effectively defends against attacks, reducing the split latency. Lastly, our replica redistribution approach incurs a minimum of 1.8 and a maximum of 2.0 seconds of overhead.

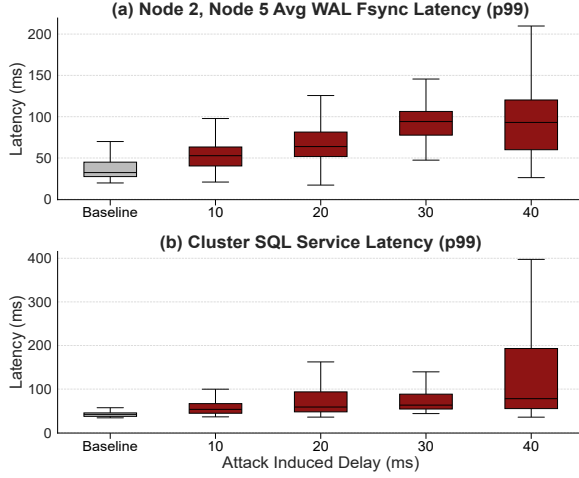


Figure 8: The gray box plot represents the baseline. (a) Average WAL fsync latency of two victim nodes (Node 2, Node 5). (b) Due to hardware contention from running dual CockroachDB instances per machine, SQL service latency doesn't increase linearly as in Figure 3 (attack triggered after 900 seconds).

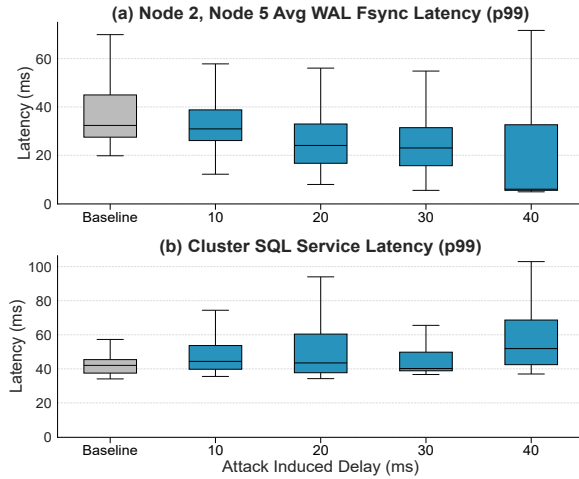


Figure 9: The gray box plot represents the baseline. (a) The two victim node WAL fsync latency is reduced by the mitigation below baseline level. (b) The cluster's SQL service latency remains within a stable range when mitigation is applied (attack triggered at 900 seconds; mitigation triggered 1 second later).

Attacks on Multiple Nodes. To prove that our replica scheme is effective not only for a single attacked node, we conducted an experiment with 6 nodes, attacking 2 nodes in a single trial run. Due to the limited number of physical servers, we co-located an additional CockroachDB instance on both the victim server and

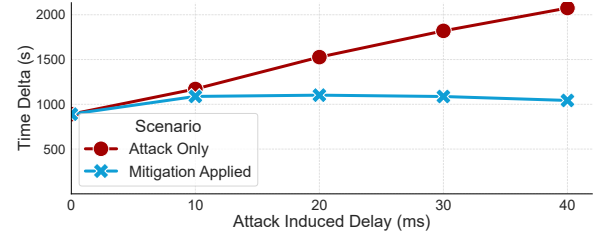


Figure 10: Scenario with two attacked nodes. The mitigation strategy decreases the split point delay (0 ms represents no attack).

the desktop. Hence, the hardware resource contention makes experiments unstable compared to the single attacked node case in Section 6.2. However, as can be seen from Figure 8, the attack cause latency increase. Figure 9 shows both WAL fsync and cluster SQL service latency mitigation. This can be seen as well in Figure 10, where our replica mitigation scheme restores the I/O throughput close to baseline even in a scenario with multiple nodes attacked. **Sequential Workloads.** CockroachDB officially discourages workloads with sequential indexing (e.g., `kv --sequential`), because all writes concentrate on the tail of the latest range [16]. This creates a single-range hotspot that prevents load-based splitting and limits throughput to a single node.

Our experiments on sequential workloads confirm this: even without attacks, we observe inherently unstable baseline performance, as the write-frontier range's replicas are non-deterministically distributed across nodes after each range split. This inherent instability further exacerbates under attack. On the victim node, the P99 WAL fsync latency degrades by up to 1,401% relative to the baseline, with degradation varying non-monotonically as attack-induced delays increase. Consequently, mitigation effectiveness ranges from -62.1% to 52.3%. Our defense strategy achieves its full effectiveness under CockroachDB's recommended settings (uniformly random key distribution).

6.3 Validation in Real-World Scenario

To validate the effectiveness of our defense in real-world conditions, we conducted a study in our tank setup shown in Figure 2. For this evaluation, we place one server in the metal enclosure and another outside of it, where it will not be influenced by the attack (on-land server). We then create a 4-node CockroachDB cluster with three nodes placed on the on-land server and one on the server in the enclosure, with our defense system implemented on the victim server. The CockroachDB KV benchmark is set with 50% reads and 50% writes for 20 minutes and logs the P99 SQL and WAL latencies for the baseline without mitigation, baseline with the mitigation, attack at 40% degradation without mitigation, and attack at 40% degradation with our framework. This yields 1200 logs for each trial. 40% degradation is equivalent to approximately 34 dB SPL above noise in our testbed. This injection volume reflects limitations of our experimental setup: the victim server achieves higher performance than the on-land server due to resource contention among nodes. As a result, to mount a successful attack, we configure the injection

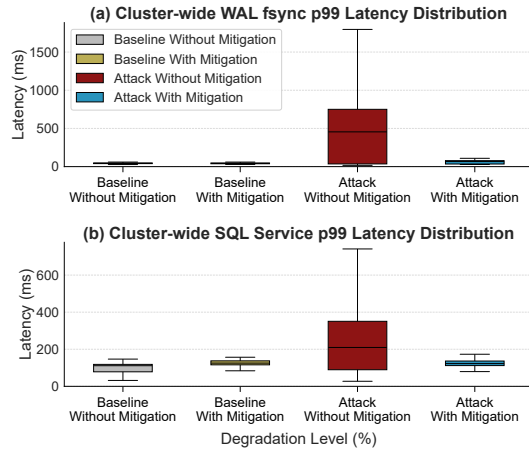


Figure 11: (a) The WAL fsync latency of the attacked node decreases to near-baseline levels when our mitigation prevents replica placement on the attacked node. (b) The overall P99 service latency decreases near baseline when the mitigation is triggered.

volume to exceed the performance of the slowest non-victim node. As shown in Figure 11, the attack raises the mean P99 SQL service latency by 212% when compared to the baseline with no mitigation. Even with this sharp increase in latency, the node’s performance generally remains within CockroachDB’s default 10-second threshold used to detect slow disks (only 1.7% of the latencies surpass this threshold) and never surpasses the 20-second threshold to remove failing nodes [16].

When mitigation is activated in response to detection, the attack induces a 25% increase in service latency, closely matching the 30% latency increase incurred by enabling mitigation alone relative to the no-mitigation baseline. This overhead occurs as a result of the increased workload pressure from redistributing leases from the node under attack to the rest of the nodes in the cluster (i.e., when 25% of the cluster is removed, the workload pressure on the remaining nodes increases accordingly). Thus, our defense can detect performance degradation that is too subtle for default thresholding mechanisms and can mitigate the effects of the attack by migrating the replicas accordingly.

7 Discussion & Future Work

Our proof-of-concept framework leverages a new detection approach and functionalities already built into standard OS and distributed databases to quickly detect and mitigate subtle acoustic attacks in underwater data centers. This allows replicas to be redirected to unaffected nodes, avoiding automatic node removal and application failure due to delays incurred when updating write-ahead logs.

Generalizability. RepliGuard leverages SQL-level placement APIs, specifically, zone constraints that partially exclude affected nodes from replica placement. This property is not unique to CockroachDB,

consequently, our mitigation approach can apply to similar architectures. For instance, TiDB [34] organizes data into regions, each replicated as an independent Raft group. On top of that, it allows configurations of label-based replica placement constraints, such as CockroachDB’s zone configurations. Similarly, YugabyteDB [82] implements per-tablet Raft groups with tablespace-based placement that controls replica distribution across zones, providing equivalent primitives for a RepliGuard-style partial offloading strategy.

Limitations and Future Works. Our evaluation is limited to CockroachDB, and the replication scheme requires at least three nodes to remain unaffected by the attack. This is a reasonable assumption since recent datacenter underwater deployments contain between 400 and 800 servers located in separate vessels [80]. Future research can focus on exploring different distributed applications, such as etcd [14] and Apache Zookeeper [5], and their behavior under attack. Finally, another limitation is our testbed, which does not fully represent real-world data center deployments.

Despite these limitations, this work provides an initial investigation and methodology to address the resiliency of underwater datacenters against acoustic attacks, particularly, the limitations of popular replica schemes in distributed databases to withstand such threats and the need for security-driven replica schemes. Future efforts can extend this by exploring tradeoffs in the strictness of consistency guarantees and resilience to physical attacks. Security-aware consensus protocols are other key areas for future study.

8 Conclusion

Previous works exploring acoustic attacks on storage systems deployed in submerged data centers propose attack detection schemes that make implicit assumptions about access to low-level hard disk drive data, which is generally inaccessible to the OS. Here, we propose a proof-of-concept framework that integrates detection and replica management, designed for quick integration with CockroachDB implementations, achieving 99.4% TPR and 99.1% TNR with a maximum approximate overhead of 0.5 seconds. Our replica management system incurs a minimum 1.8 and a maximum 2.0 seconds of overhead. This proof-of-concept analysis outlines the first step in mitigating acoustic injection attacks on distributed databases deployed in underwater data centers.

9 Data-Availability Statement

We are dedicated to promoting the principles of open science and ensuring the reproducibility of research. Research artifacts are available at <https://zenodo.org/records/19357661> [72].

Acknowledgments

The authors used Grammarly and Gemini 3 Pro to revise the text in this paper to correct any typos, grammatical errors, and awkward phrasing. This research is supported by the Office of Naval Research (ONR) grant #N000142412596

A Machine Learning Model Selection

Table 1: Summary of the acoustic attack detection performances of four models. We select the LSTM classifier for our framework.

Model	TPR	TNR	FPR	FNR
K-means Clustering	18.0%	100.0%	0.0%	81.2%
One-Class SVM	24.9%	96.0%	4.0%	75.1%
SVM Classifier	98.2%	97.8%	2.2%	1.8%
LSTM	99.4%	99.1%	0.9%	0.6%

Detection systems for acoustic attacks must balance sensitivity to subtle anomalies with robustness against false positives from normal system and workload variability. Unsupervised machine learning methods like Support Vector Machines (SVM) and K-means clustering have been extensively investigated in research for anomaly detection [10, 71]. While both of these algorithms are light-weight and fit into our runtime constraints, they rely on fixed decision boundaries over windowed features and do not explicitly model temporal dependencies or phase shifts that the acoustic attack induces [4, 61, 77]. Sequential models, particularly Long Short-Term Memory (LSTM) networks, instead can capture temporal dependencies in performance metrics, making them well-suited for detecting gradual signal variations [3, 48].

To determine the detection model for our framework, we trained and evaluated four machine learning approaches on the eBPF data described in Section 4 for different volume levels. Two of them are unsupervised: K-means clustering and One-Class SVM, and two are supervised methods: SVM Classifier and LSTM. Detection

success is measured using true positive and true negative rates. In this context, a true positive indicates the correct detection of the acoustic attack.

Results and Observations. As shown in Table 1, clustering and One-Class SVM exhibit poor attack detection performance with TPR below 25%, though both achieve high TNR, indicating they successfully identify normal system behavior but struggle to detect the lowest volumes of acoustic attacks as more similar to typical system fluctuations. On the other hand, supervised methods exhibit better performance. Both SVM and LSTM classifiers achieved over 98% TPR and over 97% TNR, with LSTM giving more accurate results overall. The LSTM model’s increased performance stems from its ability to capture the temporal characteristics inherent in acoustic attack signatures. Unlike SVM approaches that treat individual measurements as independent observations [4, 61, 77], LSTM networks maintain memory of previous system states, enabling detection of gradual signal variations that develop over time [31, 33, 63].

This temporal awareness is crucial for acoustic attacks, where performance impacts accumulate gradually and become apparent when seen as part of a data sequence. Furthermore, the variable latency patterns observed in our hybrid architecture evaluation, where SSD cache behavior creates complex, time-dependent performance signatures, align well with LSTM sequential processing capabilities. We observe that the model learns to distinguish between normal cache flushing patterns and the irregular behaviors induced by acoustic attacks. The LSTM’s recurrent architecture also provides robustness against the high variability in latency distributions we observed, as it recognizes attack patterns despite baseline fluctuations. Based on these observations we decide to use an LSTM classifier as our detection algorithm.

References

- [1] Adnan Abdullah, David Blow, Sara Rampazzi, and Md Jahidul Islam. 2025. Active Localization of Close-range Adversarial Acoustic Sources for Underwater Data Center Surveillance. *arXiv preprint arXiv:2510.20122* (2025).
- [2] Marcelo Abranches, Oliver Michel, Eric Keller, and Stefan Schmid. 2021. Efficient network monitoring applications in the kernel with ebpf and xdp. In *2021 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. IEEE, 28–34.
- [3] Abdalaziz R Al-Maeeni, Aziz Temirkhanov, Artem Ryzhikov, and Mikhail Hushchyn. 2025. Performance Modeling of Data Storage Systems Using Generative Models. *IEEE Access* 13 (2025), 49643–49658.
- [4] Mennatallah Amer, Markus Goldstein, and Slim Abdennadher. 2013. Enhancing one-class support vector machines for unsupervised anomaly detection. In *Proceedings of the ACM SIGKDD Workshop on Outlier Detection and Description* (Chicago, Illinois) (ODD '13). Association for Computing Machinery, New York, NY, USA, 8–15. doi:10.1145/2500853.2500857
- [5] Apache Software Foundation. 2026. Apache ZooKeeper: A Distributed Coordination Service for Distributed Applications. <https://zookeeper.apache.org/>.
- [6] Jens Axboe. [n.d.]. Flexible I/O Tester. <https://github.com/axboe/fio>. Accessed: 2025-09-15.
- [7] Ralph E Blake. 1961. Basic vibration theory. *Shock and vibration handbook* 1 (1961), 2–8.
- [8] David Blow, Adnan Abdullah, Jennifer Sheldon, Weidong Zhu, Sara Rampazzi, and Md Jahidul Islam. 2025. Detection and localization of acoustic vulnerabilities of underwater data centers for remote surveillance. In *Ocean Sensing and Monitoring XVII*, Vol. 13482. SPIE, 72–82.
- [9] Connor Bolton, Sara Rampazzi, Chao hao Li, Andrew Kwong, Wenyan Xu, and Kevin Fu. 2018. Blue note: How intentional acoustic interference damages availability and integrity in hard disk drives and operating systems. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1048–1062.
- [10] Polyzois Bountzlis, Dimitris Kavallieros, Theodora Tsikrika, Stefanos Vrochidis, and Ioannis Kompatsiaris. 2025. A deep one-class classifier for network anomaly detection using autoencoders and one-class support vector machines. *Frontiers in Computer Science* 7 (2025), 1646679.
- [11] Cyril Cassagnes, Lucian Trestioreanu, Clement Joly, and Radu State. 2020. The rise of eBPF for non-intrusive performance monitoring. In *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 1–7.
- [12] Jack Clark, Alastair F. Donaldson, John Wickerson, and Manuel Rigger. 2024. Validating Database System Isolation Level Implementations with Version Certificate Recovery. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*. ACM, 1–15. doi:10.1145/3627703.3650080
- [13] Clark Synthesis. [n.d.]. AQ339 Diluvio Underwater Loudspeaker. <https://clarksynthesis.com/aq339/>.
- [14] Cloud Native Computing Foundation. 2026. etcd: A highly-available key value store. <https://etcd.io/>.
- [15] Cockroach Labs. 2025. ACID database. <https://www.cockroachlabs.com/glossary/distributed-db/acid-database/>.
- [16] Cockroach Labs. 2025. CockroachDB. <https://www.cockroachlabs.com/docs/stable>. Accessed: 2025-09-16.
- [17] Cockroach Labs. 2025. Replication Layer: Load-Based Replica Rebalancing. <https://www.cockroachlabs.com/docs/v25.4/architecture/replication-layer#load-based-replica-rebalancing>.
- [18] Cockroach Labs. 2026. Admission Control. <https://www.cockroachlabs.com/docs/stable/admission-control>.
- [19] Cockroach Labs. 2026. Load-Based Splitting. <https://www.cockroachlabs.com/docs/stable/load-based-splitting>.
- [20] CockroachDB. 2015. The new stack: Meet CockroachDB, the resilient SQL database. <https://www.cockroachlabs.com/blog/the-new-stack-meet-cockroachdb-the-resilient-sql-database/#:~:text=With%20companies%20like%20SpaceX%2C%20and,%2C%20resilience%2C%20and%20data%20locality>.
- [21] CockroachDB. 2023. The history of databases at Netflix: From Cassandra to CockroachDB. <https://www.cockroachlabs.com/blog/netflix-at-cockroachdb/>.
- [22] James C Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, Jeffrey John Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. 2013. Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems (TOCS)* 31, 3 (2013), 1–22.
- [23] CouchBase. [n.d.]. Cross Data Center Replication (XDCR). <https://docs.couchbase.com/server/current/learn/clusters-and-availability/xdcr-overview.html>. Accessed: 2025-09-16.
- [24] CouchBase. n.d. Intra-Cluster Replication. <https://docs.couchbase.com/server/current/learn/clusters-and-availability/intra-cluster-replication.html>. Accessed: 2025-09-16.
- [25] eBPF. [n.d.]. eBPF on Linux. <https://docs.ebpf.io/linux/>. Accessed: 2025-09-15.
- [26] Facebook, Inc. 2025. IO Controller. <https://facebookmicrosites.github.io/cgroup2/docs/io-controller.html>. Accessed: 2025-09-16.
- [27] Maya Fawaz. 2025. Hays County residents aren't happy about a new neighbor moving in: a data center. <https://www.kut.org/energy-environment/2025-06-04/hays-county-texas-data-center-ai>.
- [28] Florevia. [n.d.]. Walfront Resonance Speaker 50mm Vibration Speaker Vibrating Speaker Speaker 2inch All Frequency Resonance Louder Speaker Exciter Speaker (4 ohm 25W). <https://www.floreviaboutique.com/product-p-456919.html>. Accessed: 2026-01-27.
- [29] Genasys. [n.d.]. Long Range Acoustic Devices. <https://genasys.com/lrad-products/>. Accessed: 2025-09-16.
- [30] Gooding, Matthew. 2024. China's HiCloud launches wind-powered underwater data center, targets 500MW subsea deployment. <https://www.datacenterdynamics.com/en/news/chinas-hicloud-launches-wind-powered-underwater-data-center-targets-500mw-subsea-deployment/#:~:text=The%20demonstration%20project%20marks%20the,the%20North%20Sea%20in%202018>. Accessed: 2025-11-19.
- [31] Yunfei Gu, Chentao Wu, and Xubin He. 2024. Exploit both SMART Attributes and NAND Flash Wear Characteristics to Effectively Forecast SSD-based Storage Failures in Clusters. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 1101–1117.
- [32] Herreria, Anne. 2024. A Balancing Act: HDDs and SSDs in Modern Data Centers. <https://blog.westerndigital.com/a-balancing-act-hdds-and-ssds-in-modern-data-centers/>. Accessed: 2025-11-19.
- [33] Lihan Hu, Lixin Han, Zhenyuan Xu, Tianming Jiang, and Huijun Qi. 2020. A disk failure prediction method based on LSTM network due to its individual specificity. *Procedia Computer Science* 176 (2020), 791–799.
- [34] Dongxu Huang, Qiu Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, et al. 2020. TiDB: a Raft-based HTAP database. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3072–3084.
- [35] Patrick Hunt, Mahadev Konar, Flavio P Junqueira, and Benjamin Reed. 2010. {ZooKeeper}: Wait-free coordination for internet-scale systems. In *2010 USENIX Annual Technical Conference (USENIX ATC 10)*.
- [36] IBM. [n.d.]. What is data replication? <https://www.ibm.com/think/topics/data-replication>. Accessed: 2025-09-15.
- [37] Isovalent. [n.d.]. What is Cilium? <https://cilium.io/get-started/>. Accessed: 2025-09-15.
- [38] Peter Judge. 2022. Subsea Cloud announces three underwater data center projects. <https://www.datacenterdynamics.com/en/news/subsea-cloud-announces-three-underwater-data-center-projects/>.
- [39] Judge, Peter. 2024. China's HiCloud reports commercial underwater data center modules are working. <https://www.datacenterdynamics.com/en/news/chinas-highlander-completes-first-commercial-underwater-data-center-looks-for-exports/>. Accessed: 2025-11-19.
- [40] Laila Kearney. 2025. America's largest power grid is struggling to meet demand from AI. <https://www.reuters.com/sustainability/boards-policy-regulation/america-largest-power-grid-is-struggling-meet-demand-ai-2025-07-09/>.
- [41] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [42] Joe Kohut. 2025. Residents rally against data center zoning proposals in Lackawanna County communities. <https://www.wnep.com/article/news/local/lackawanna-county/data-center-lackawanna-county-archbald-blakely-jessup-council-votes/523-627d1d15-f8e2-436e-bc39-732ec3e3b26>.
- [43] Cockroach Labs. [n.d.]. cockroach workload. <https://www.cockroachlabs.com/docs/stable/cockroach-workload#kv-workload>.
- [44] Qiang Li, Qiao Xiang, Yuxin Wang, Hao hao Song, Ridi Wen, Wenhui Yao, Yuanyuan Dong, Shuqi Zhao, Shuo Huang, Zhaosheng Zhu, et al. 2023. More than capacity: Performance-oriented evolution of pangu in alibaba. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. 331–346.
- [45] Ruiming Lu, Yunchi Lu, Yuxuan Jiang, Guangtao Xue, and Peng Huang. 2025. {One-Size-Fits-None}: Understanding and Enhancing {Slow-Fault} Tolerance in Modern Distributed Systems. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 359–378.
- [46] Ruiming Lu, Erci Xu, Yiming Zhang, Fengyi Zhu, Zhaosheng Zhu, Mengtian Wang, Zongpeng Zhu, Guangtao Xue, Jiwei Shu, Minglu Li, et al. 2023. Perseus: A Fail-Slow detection framework for cloud storage systems. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. 49–64.
- [47] Ruiming Lu, Erci Xu, Yiming Zhang, Zhaosheng Zhu, Mengtian Wang, Zongpeng Zhu, Guangtao Xue, Minglu Li, and Jiesheng Wu. 2022. NVMe SSD failures in the field: the {Fail-Stop} and the {Fail-Slow}. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 1005–1020.
- [48] Joshua Ludolf, Yesmin Reyna-Hernandez, and Matthew Trevino. 2024. Reproduction Research of FSA-Benchmark. *arXiv preprint arXiv:2501.14739* (2024).
- [49] Peter Mattis. 2018. perf: pacing user writes, flushes and compactions (GitHub Issue #7). <https://github.com/cockroachdb/pebble/issues/7>.
- [50] Peter Mattis. 2020. Introducing Pebble: A RocksDB-inspired key-value store written in Go. <https://www.cockroachlabs.com/blog/pebble-rocksdb-kv-store/>.
- [51] Changwoo Min, Kangnyeon Kim, Hyunjin Cho, Sang-Won Lee, and Young Ik Eom. 2012. SFS: random write considered harmful in solid state drives. In *USENIX FAST*.
- [52] MitXPC. [n.d.]. ASRock Rack 1U4LW-X570/2L2T RPSU AM4 1U Server, Dual 10GbE LAN, Dual GbE LAN, IPMI. <https://mitxpc.com/products/1u4lw-x570-2l2t-rpsu>.

- [53] Sebastian Moss. [n. d.]. China deploys 1,400-ton commercial underwater data center. <https://www.datacenterdynamics.com/en/news/china-deploys-1400-ton-commercial-underwater-data-center/>.
- [54] Subramanian Muralidhar, Wyatt Lloyd, Sabyasachi Roy, Cory Hill, Ernest Lin, Weiwen Liu, Satadru Pan, Shiva Shankar, Viswanath Sivakumar, Linpeng Tang, and Sanjeev Kumar. 2014. f4: Facebook's Warm BLOB Storage System. In *USENIX OSDI*.
- [55] Ryan Murphy and Emily Feng. 2025. Why more residents are saying 'No' to AI data centers in their backyard. <https://www.npr.org/2025/07/17/nx-s1-5469933/virginia-data-centers-residents-saying-no>.
- [56] NetworkOcean. 2024. NetworkOcean. <https://networkocean.io/>.
- [57] Khiem Ngo, Siddhartha Sen, and Wyatt Lloyd. 2020. Tolerating slowdowns in replicated state machines using copilots. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 583–598.
- [58] Diego Ongaro and John Ousterhout. 2014. In search of an understandable consensus algorithm. In *2014 USENIX annual technical conference (USENIX ATC 14)*. 305–319.
- [59] Open-CAS. 2026. Open-CAS: Open Cache Acceleration Software. <https://open-cas.com/index.html>.
- [60] Biswaranjan Panda, Deepthi Srinivasan, Huan Ke, Karan Gupta, Vinayak Khot, and Haryadi S Gunawi. 2019. {IASO}: A {Fail-Slow} detection and mitigation framework for distributed storage services. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. 47–62.
- [61] Guansong Pang, Chunhua Shen, Longbing Cao, and Anton Van Den Hengel. 2021. Deep learning for anomaly detection: A review. *ACM computing surveys (CSUR)* 54, 2 (2021), 1–38.
- [62] Alastair Robertson. [n. d.]. bpftrace. <https://bpftrace.org/>. Accessed: 2025-09-15.
- [63] Snehanishu Saha, Jyotirmoy Sarkar, Soma Dhavala, Santonu Sarkar, and Preyank Mota. 2023. Quantile LSTM: A Robust LSTM for Anomaly Detection In Time Series Data. *arXiv preprint arXiv:2302.08712* (2023).
- [64] Seagate. [n. d.]. Exos 7E2 Datasheet. https://www.seagate.com/www-content/datasheets/pdfs/exos-7-e2DS1956-1-1709US-en_US.pdf. Accessed: 2025-09-15.
- [65] Seagate. 2024. Why HDDs Dominate Hyperscale Cloud Architecture . <https://www.seagate.com/blog/why-hdds-dominate-hyperscale-cloud-architecture/>. Accessed: 2025-11-19.
- [66] Seagate. [n.d.]. Maximize Security, Lock Down Hard Drive Firmware with Seagate Secure Download & Diagnostics. <https://www.seagate.com/files/www-content/solutions-content/security-and-encryption/en-us/docs/seagate-secure-download-diagnostics-with-maximize-sec-lock-down-hard-drive-firmware-tp684-1-1508us.pdf>. Accessed: 2026-02-02.
- [67] Seagate Technology. 2024. Seagate's Data Center Workloads Served Entirely By Hybrid And Hard Drive Storage Systems. <https://www.seagate.com/blog/seagate-s-data-center-workloads-served-entirely-by-hybrid-and-hard-drive-storage-systems/>.
- [68] Paula Seligson. 2026. The \$3 Trillion AI Data Center Build-Out Becomes All-Consuming For Debt Markets. <https://finance.yahoo.com/news/3-trillion-ai-data-center-110030774.html>.
- [69] Mohammad Shahradd, Arsalan Mosenia, Liwei Song, Mung Chiang, David Wentzlaff, and Prateek Mittal. 2018. Acoustic denial of service attacks on hard disk drives. In *ACM ASHES*.
- [70] Ali Shakarami, Mostafa Ghobaei-Arani, Ali Shahidinejad, Mohammad Masdari, and Hamid Shakarami. 2021. Data replication schemes in cloud computing: a survey. *Cluster Computing* 24, 3 (2021), 2545–2579.
- [71] K Shanthi and R Maruthi. 2023. Machine Learning Approach for Anomaly-Based Intrusion Detection Systems Using Isolation Forest Model and Support Vector Machine. In *2023 5th International Conference on Inventive Research in Computing Applications (ICIRCA)*. IEEE, 136–139.
- [72] Jennifer Sheldon, Yungwoo Ko, Sri Hrushikesh Varma Bhupathiraju, Sara Osmanovic, Weidong Zhu, Md Jahidul Islam, and Sara Rampazzi. 2026. Reproduction Package for RepliGuard: Policy-Driven Replica Management Framework for Protecting against Acoustic Attacks. doi:10.5281/zenodo.19357661
- [73] Jennifer Sheldon, Weidong Zhu, Adnan Abdullah, Sri Hrushikesh Varma Bhupathiraju, Takeshi Sugawara, Kevin RB Butler, Md Jahidul Islam, and Sara Rampazzi. 2024. Aquasonic: Acoustic manipulation of underwater data center operations and resource management. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 331–349.
- [74] Jennifer Sheldon, Weidong Zhu, Adnan Abdullah, Kevin Butler, Md Jahidul Islam, and Sara Rampazzi. 2023. Deep note: Can acoustic interference damage the availability of hard disk storage in underwater data centers?. In *Proceedings of the 15th ACM Workshop on Hot Topics in Storage and File Systems*. 51–57.
- [75] SuperMicro. [n. d.]. 2U Chassis. <https://www.supermicro.com/products/chassis/2u/?chs=823>.
- [76] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieser, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, Paul Bardea, Amruta Ranade, Ben Darnell, Bram Gruneir, Justin Jaffray, Lucy Zhang, and Peter Mattis. 2020. CockroachDB: The Resilient Geo-Distributed SQL Database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. 1493–1509. doi:10.1145/3318464.3386134
- [77] David MJ Tax and Robert PW Duin. 2004. Support vector data description. *Machine learning* 54, 1 (2004), 45–66.
- [78] The Economic Times. 2025. AI data centers to drive 11-fold rise in water consumption by 2028: Morgan Stanley. <https://economictimes.indiatimes.com/tech/artificial-intelligence/ai-data-centers-to-drive-11-fold-rise-in-water-consumption-by-2028-morgan-stanley/articleshow/123755912.cms?from=mdr>.
- [79] The Linux Kernel Organization. 2026. Device Mapper. <https://docs.kernel.org/admin-guide/device-mapper/index.html>.
- [80] You Xiaoying. 2025. China Is Putting Data Centers in the Ocean to Keep Them Cool. <https://www.scientificamerican.com/article/china-powers-ai-boom-with-undersea-data-centers/>.
- [81] Tzu-Wei Yang, Seth Pollen, Mustafa Uysal, Arif Merchant, and Homer Wolfmeister. 2022. CacheSack: Admission Optimization for Google Datacenter Flash Caches. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 1021–1036.
- [82] Yugabyte. 2026. YugabyteDB Documentation. <https://docs.yugabyte.com/>.

Received 2026-02-05; accepted 2026-03-19